Web Development using PHP

PHP Loops, Arrays

(Lecture 3)

Prepared By: BITS IT Team

Venue: GS Training Room

Table of Contents

1. PHP Loops 3
1.1 The while Loop 3
Example 1.1 3
1.2 The do...while Statement 4
Example 1.2 4
2. PHP Arrays 5
2.1 Numeric Arrays 5
Example 2.1 6
2.2 Associative Arrays 6
Example 2.2 6
Example 2.3 6
2.3 Multidimensional Arrays 7
Example 2.4 7
Example 2.5 8
2.4 The for Loop 8
Example 2.6 8
2.5 The foreach Loop 9
Example 2.7 9
Example 2.8 10
3 Resources 11
3.1 websites 11
3.2 Books 11

1. PHP Loops

Loops execute a block of code a specified number of times, or while a specified condition is true.

Often when you write code, you want the same block of code to run over and over again in a row. Instead of adding several almost equal lines in a script we can use loops to perform a task like this.

In PHP, we have the following looping statements:

- **while** - loops through a block of code while a specified condition is true
- **do...while** - loops through a block of code once, and then repeats the loop as long as a specified condition is true
- **for** - loops through a block of code a specified number of times
- **foreach** - loops through a block of code for each element in an array

1.1 The while Loop

The while loop executes a block of code while a condition is true.

Syntax

```
while (condition)
 {
 code to be executed;
 }
```

Example 1.1

The example below defines a loop that starts with i=1. The loop will continue to run as long as i is less than, or equal to 5. i will increase by 1 each time the loop runs:

```
<html>
<body>
<?php
$i=1;
while($i<=5)
 {
 echo "The number is " . $i . "<br />";
 $i++;
 }
?>
</body>
</html>
```

Output:

```
The number is 1
The number is 2
The number is 3
The number is 4
The number is 5
```

1.2 The do...while Statement

The do...while statement will always execute the block of code once, it will then check the condition, and repeat the loop while the condition is true.

Syntax

```
do
 {
 code to be executed;
 }
while (condition);
```

Example 1.2

The example below defines a loop that starts with i=1. It will then increment i with 1, and write some output. Then the condition is checked, and the loop will continue to run as long as i is less than, or equal to 5:

```
<html>
<body>
<?php
$i=1;
do
 {
 $i++;
 echo "The number is " . $i . "<br />";
 }
while ($i<=5);
?>
</body>
</html>
```

Output:

```
The number is 2
The number is 3
The number is 4
The number is 5
The number is 6
```

2. PHP Arrays

An array stores multiple values in one single variable.

If you have a list of items (a list of car names, for example), storing the cars in single variables could look like this:

```
$cars1="Toyota";
$cars2="Hyundai";
$cars3="BMW";
```

However, what if you want to loop through the cars and find a specific one? And what if you had not 3 cars, but 300?

The best solution here is to use an array!

An array can hold all your variable values under a single name. And you can access the values by referring to the array name.

Each element in the array has its own index so that it can be easily accessed.

In PHP, there are three kinds of arrays:

- **Numeric array** - An array with a numeric index
- **Associative array** - An array where each ID key is associated with a value
- **Multidimensional array** - An array containing one or more arrays

2.1 Numeric Arrays

A numeric array stores each array element with a numeric index.

There are two methods to create a numeric array.

1. In the following example the index are automatically assigned (the index starts at 0):

```
$cars=array("Saab","Volvo","BMW","Toyota");
```

2. In the following example we assign the index manually:

```
$cars[0]="Saab";
$cars[1]="Volvo";
$cars[2]="BMW";
$cars[3]="Toyota";
```

Example 2.1

In the following example you access the variable values by referring to the array name and index:

```
<?php
$cars[0]="Saab";
$cars[1]="Volvo";
$cars[2]="BMW";
$cars[3]="Toyota";
echo $cars[0] . " and " . $cars[1] . " are Swedish cars.";
?>
```

The code above will output:

```
Saab and Volvo are Swedish cars.
```

2.2 Associative Arrays

An associative array, each ID key is associated with a value.

When storing data about specific named values, a numerical array is not always the best way to do it.

With associative arrays we can use the values as keys and assign values to them.

Example 2.2

In this example we use an array to assign ages to the different persons:

```
$ages = array("A"=>32, "B"=>30, "C"=>34);
```

Example 2.3

This example is the same as example 2.2, but shows a different way of creating the array:

```
$ages['A'] = "32";
```

```
$ages['B'] = "30";
$ages['C'] = "34";
```

The ID keys can be used in a script:

```
<?php
$ages['A'] = "32";
$ages['B'] = "30";
$ages['C'] = "34";

echo "A is " . $ages['B'] . " years old.";
?>
```

The code above will output:

```
A is 32 years old.
```

2.3 Multidimensional Arrays

In a multidimensional array, each element in the main array can also be an array. And each element in the sub-array can be an array, and so on.

Example 2.4

In this example we create a multidimensional array, with automatically assigned ID keys:

```
$shops = array
 (
 "flowers"=>array( "rose", "daisy", "orchid"  ),
 "prices"=>array  (  "1.5",  "0.75",  "1.15"  )
 );
```

The array above would look like this if written to the output:

```
Array
(
   [flowers] => Array
      (
         [0] => rose
         [1] => daisy
         [2] => orchid
      )

   [prices] => Array
      (
         [0] => 1.5
         [1] => 0.75
```

```
        [2] => 1.15
    )

)
```

Example 2.5

Lets try displaying a single value from the array above:

```
echo $shops['flowers'][0] .' costs '. $shops['prices'][0];
```

The code above will output:

```
rose costs 1.5
```

2.4 The for Loop

The for loop is used when you know in advance how many times the script should run.

Syntax

for (*init; condition; increment*)
 {
 code to be executed;
 }

Parameters:

- *init*: Mostly used to set a counter (but can be any code to be executed once at the beginning of the loop)
- *condition*: Evaluated for each loop iteration. If it evaluates to TRUE, the loop continues. If it evaluates to FALSE, the loop ends.
- *increment*: Mostly used to increment a counter (but can be any code to be executed at the end of the loop)

Note: Each of the parameters above can be empty, or have multiple expressions (separated by commas).

Example 2.6

The example below defines a loop that starts with i=1. The loop will continue to run as long as i is less than, or equal to 5. i will increase by 1 each time the loop runs:

```
<html>
<body>
```

```
<?php
for ($i=1; $i<=5; $i++)
 {
 echo "The number is " . $i . "<br />";
 }
?>
</body>
</html>
```

Output:

```
The number is 1
The number is 2
The number is 3
The number is 4
The number is 5
```

2.5 The foreach Loop

The foreach loop is used to loop through arrays.

Syntax

foreach ($*array* as $*value*)
 {
 code to be executed;
 }

For every loop iteration, the value of the current array element is assigned to $value (and the array pointer is moved by one) - so on the next loop iteration, you'll be looking at the next array value.

Example 2.7

The following example demonstrates a loop that will print the values of the given array:

```
<html>
<body>
<?php
$x=array("one","two","three");
foreach ($x as $value)
 {
 echo $value . "<br />";
 }
?>
</body>
</html>
```

Output:

```
one
two
three
```

Displaying array (from example 2.5) using for loop;

Example 2.8

```
<html>
<body>
<?php
echo "<h2>Manual access to each element from associative array</h2>";

for ($row = 0; $row < 3; $row++)
{
  echo $shops['flowers'][$row] .' costs '. $shops['prices'][$row];
   echo "<br />";
}
?>
</body>
</html>
```

3 Resources

3.1 websites

http://www.w3schools.com/

3.2 Books

Professional PHP Programming by Wrox word press Ltd. **ISBN:** 81-7366-201-0